

自前 malloc — ヒープを自分で管理する

今日のゴール

`malloc` を自分で書く。バンプ割り当て（切り出すだけ）から始め、`free` した領域を再利用できるところまで作る。コマ 13 で作った連結リストが、**自前のアロケータ**の上で動くことを確認する。

1 この回の位置づけ

- ・ランタイム発展シリーズの第 3 回（選択制）。前提はコマ 13（`malloc` と連結リスト）
- ・書くのはすべて Core プロファイルの C。自分のコンパイラでコンパイルする
- ・コマ 13 の資料で「`malloc` の仕組みは実装しない」と言った、その中身を作る
- ・R1・R2 とは独立している（`libc` は使ってよい。`malloc` だけ自分のものに置き換える）

2 malloc は何をしているのか

コマ 13 では、こう説明した。

`malloc` は、指定されたバイト数ぶんのメモリ領域をヒープから確保し、その先頭アドレスを返すライブラリ関数である。

ではその「確保」とは何か。大きな領域を、要求されたぶんずつ切り分けて配ることである。配った場所を覚えておかないと、次の要求で同じ場所を二重に配ってしまう。

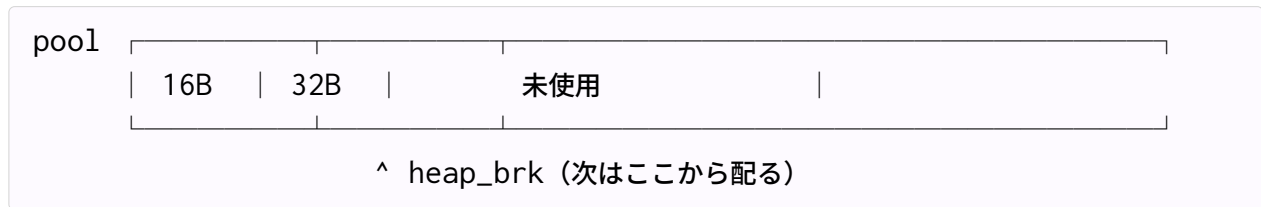
この回では、ヒープの元になる領域（プール）を呼び出し側から渡してもらう。

```
char pool[1024];
heap_init(pool, 1024);
```

本物の `malloc` は OS から `brk` / `mmap` で領域をもらうが、そこは R1 のシステムコールの話で、割り当ての本質ではない。

3 段階 1: バンプ割り当て —ただ切り出す

いちばん単純な方式は、「次に配る位置」を 1 つ覚えておき、要求されたぶんだけ進めることである。

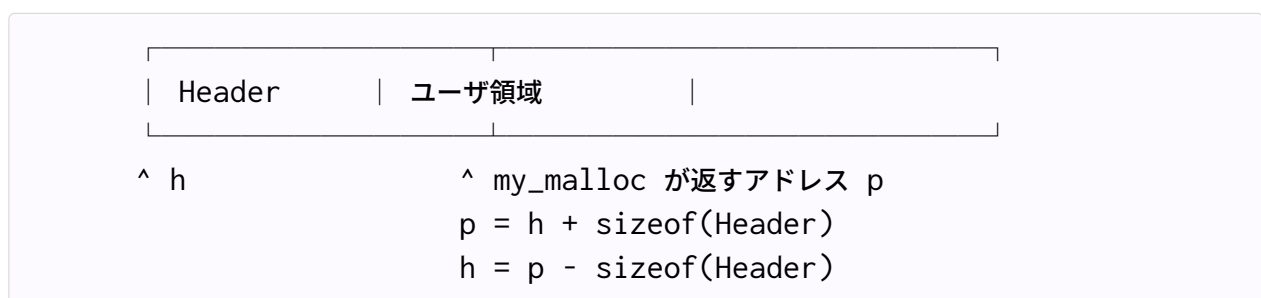


`heap_brk` を進めるだけなので非常に速い。しかし `free` ができない（真ん中を返されても、`heap_brk` を戻せない）。それでもコンパイラのような「起動して、確保しまくって、終わったら全部捨てる」プログラムには十分で、実際に [workbook/porting/README.md](#) の C 実装の規約でも「`free` は不要」としている。

4 段階 2: ヘッダを付けて再利用する

`free` を作るには、返された領域がどれだけの大きさだったかを知る必要がある。`my_free(p)` に渡されるのはアドレスだけだからである。

そこで、配る領域の手前に管理用のヘッダを置く。



ヘッダには、大きさ・解放済みフラグ・次のブロックへのポインタを持たせる。コマ 13 で作った自己参照構造体そのものである。

```
typedef struct Header {
    int size;
    int free;
    struct Header *next;
} Header;
```

これで、`my_free(p)` は「`p` の手前のヘッダを見て `free = 1` にする」だけでよい。`my_malloc` は、まず管理リストから「解放済みで十分な大きさ」のブロックを探し (**first fit**)、あればそれを再利用する。なければ未使用領域から切り出す。

補足

アドレスの計算にキャストが使えない。本物の C なら `(char *)h + sizeof(Header)` と書くところだが、Core プロファイルにキャストはない (`language_spec.md` の除外一覧)。代わりに、いったん `char *` の変数に代入してからポインタ演算する。`char *` の演算は 1 バイト単位なので、`sizeof(Header)` を足せば期待どおりに進む (コマ 10 のポインタ演算の型サイズの話)。

5 8 バイト境界に揃える

要求サイズは 8 の倍数に切り上げる (`align8`)。コマ 7・コマ 13 で見たように、`int` は 4 バイト境界、ポインタは 8 バイト境界に置かれる必要があり、揃っていないアドレスを返すと `Bus error` や無音の誤動作になるからである。「アラインメントを揃える」のは、フレームサイズのとおり考え方である。

6 実装

Step	実装対象	内容
1	<code>align8 / heap_init / my_malloc</code> (切り出し)	バンク割り当て
2	<code>find_free / my_free / my_malloc</code> (再利用)	ヘッダとフリーリスト
3	<code>heap_used</code>	使用量の観察

Step 1 だけでも `tests/list_build.c` は通る。Step 2 まで進むと `tests/reuse.c` (解放した跡地が再利用される) が通る。

7 テスト

```
python3 check.py
```

テスト	確認内容
<code>list_build.c</code>	自前 <code>malloc</code> で連結リストが作れる（コマ 13 の再現）
<code>reuse.c</code>	解放したブロックが同じアドレスで再利用される
<code>exhaust.c</code>	プールを使い切ったら 0（NULL）を返して止まる

`exhaust.c` は「無限に確保できてしまう」バグ（終端チェック忘れ）を検出する。

重要

プールは **1024 バイト程度にとどめる**。大きなローカル配列（4096 バイトなど）を宣言すると、スタックフレームのオフセットが RV64 の `addi` 命令の即値の範囲（12bit = -2048 ~ 2047）を超え、アセンブラがエラーを出す。これは自作コンパイラの実際の制限で、発展課題で直せる。

8 発展課題

1. **12bit の壁を越える**: 大きなフレームでも動くよう、オフセットが範囲外のときは `li` で一度レジスタに入れてから `add` する コード生成に直す（コマ 4・コマ 7 のプロローグとローカル変数アクセス）
2. **分割（split）**: 100 バイトの空きブロックに 16 バイトの要求が来たとき、いまは 100 バイト丸ごと使ってしまう。余りを新しい空きブロックに分ける
3. **併合（coalesce）**: 隣り合う空きブロックを 1 つにまとめる。さもないと、小さな空きだらけになって大きな要求が通らなくなる（断片化）
4. **best fit にする**: `find_free` を「最初に見つかったもの」ではなく「要求に最も近い大きさのもの」を返す方式に変え、断片化の減り方を比べる
5. **libc の malloc と差し替えて計測**: 確保・解放を何万回も繰り返すプログラムを書き、自前版と libc 版で実行時間を比べる

補足

コラム: アロケータは奥が深い。実用の `malloc` は、サイズ別のフリーリスト (bins)、スレッドごとのアリーナ、小さい要求専用の高速 path、OS への返却など、多層の工夫でできている。それでも中心にあるのは、いま書いた「**ヘッダで大きさを覚え、空きを探して配る**」という考え方である。ガベージコレクションを持つ言語 (Java や Go) では、この「空きを探す」部分に「もう誰も指していない領域を見つける」処理が加わる。